

Day 9: Memory Segments & Linker Scripts

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

.text Section — Code in Flash

`.text` contains all compiled machine code instructions. It lives in **Flash** (non-volatile, 1 MB on nRF52840). After reset, the CPU starts executing from the reset handler address, which is in `.text`. Flash is much slower to write than RAM but retains data without power.

.rodata Section — Read-Only Data in Flash

`const` global variables, string literals, and `constexpr` arrays go into `.rodata`. This section also lives in Flash — **no RAM is used** for these values. This is why you should make your lookup tables `constexpr const` rather than just global arrays.

.data Section — Initialized Globals in RAM

Global and static variables with **non-zero initial values** go into `.data`. The initial values are stored in Flash, and the startup code (`crt0` /reset handler) copies them from Flash into RAM before `main()` runs:

```
static uint32_t config = 0xDEADBEEFUL; // .data - value copied Flash→RAM at boot
```

.bss Section — Zero-Initialized Globals

Global and static variables with **zero initial values** (or uninitialized) go into `.bss`. They are not stored in Flash at all — the startup code just zeroes this region of RAM:

```
static uint32_t counter;          // .bss – zeroed by startup code, no Flash cost
static bool initialized = false; // .bss – false is zero
```

Stack — Local Variables and Call Frames

The **stack** grows **downward** from the top of RAM. Each function call pushes a **stack frame** containing: return address, saved registers, and local variables. On nRF52840 with 256KB RAM, Zephyr allocates a separate stack for each thread (configured with `K_THREAD_STACK_DEFINE`).

Stack overflow is a common embedded bug — use `CONFIG_STACK_SENTINEL=y` to detect it.

__noinit — Survives Soft Resets

Variables in the `.noinit` section are **not zeroed** during startup. This lets you preserve data across software resets (but not power cycles):

```
static uint32_t boot_counter __attribute__((section(".noinit")));
// Increments every reset – useful for crash counting, boot loop detection
```

Zephyr Linker Script

Zephyr's linker script (`zephyr/kernel/include/linker/linker.ld`) defines all memory regions and section placements. After building, examine the map file:

```
cat build/zephyr/zephyr.map | grep " .data"
```

west build Map File Analysis

The linker map file (`build/zephyr/zephyr.map`) shows every symbol: its section, address, and size. Key uses:

- Find what's consuming RAM

- Verify a constant is in Flash (.rodata) not RAM (.data)
 - Debug unexpected large code size
-

Practice Tasks

1. Build any day's example and open `build/zephyr/zephyr.map` — find your global variables
 2. Add a large `const uint8_t table[1024]` — confirm it's in Flash, not RAM, via the map file
 3. Add a `.noinit` boot counter — reset the board 5 times and watch it increment
 4. Use `CONFIG_STACK_SENTINEL=y` and intentionally overflow a thread stack to see the error
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day09
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day09/main.cpp](#)